JavaScript講座【レベル1:入門編】
～ゼロから始めるJavaScriptの基本と仕組み～

2026/03/03

Kazuma Sekiguchi

自己紹介

関口和真

株式会社コムセントCTO
Webシステム開発、スマートフォンアプリ制作、
サーバー構築、運用など

スマートフォンを使った動画配信アプリの制作

サーバーサイドシステムの作成

フロントエンド部分の作成

目標

- JavaScriptの基礎が理解できる
- JavaScriptを利用してHTML＋CSSで作成されたコンテンツの見た目を変更できるようになる

今回のAgenda

- プログラミングとは何か（入力→処理→出力、状態と副作用）
- JavaScriptの特徴（ブラウザ、動的型付け、イベント駆動）
- JSの記述方法（script、defer、Consoleで試す）
- DevToolsの基本（Console/Sources、エラーの読み方：行番号・スタック）
- 変数・数値・文字列、テンプレート文字列、演算子、比較

使用するソフトウェア

- 今回利用するのはVisual Studio Codeとプラグイン
- ブラウザーはGoogleChrome

- JavaScript自体はエディタで作成可能

プログラミングとは

- ある問題や時間の掛かることをPCやコンピュータに代わりにやらせるためにコンピュータ上で行なう処理を特定の記法に沿って記述したもの
  - 文字の羅列である
- 本来コンピュータは人間の言葉は理解しないため、機械語という言葉で表す必要がある
  - 機械語は人間には理解しづらい
  - 0と1の羅列である
- 機械語と人間の言葉を仲介するためのものがプログラミング言語

プログラミング言語とは

- 人間の言葉に近いものを使って、記述していくための言語
  - 言語とあるように言葉である
  - ほとんどが英語、的なもので記述する
  - 最終的にプログラミング言語で記述されたものを変換して、機械語にした上でコンピュータ上で動かす
- 用途にあわせて最適なプログラミング言語は異なる
  - 用途に合わせたプログラミング言語を選択する
    - C言語、C++言語：PCなどで利用されるアプリケーションを作成するために使われる。高速に動作する
    - Python言語：AIなどで良く使われる。統計データなどを扱うのが得意。速度は若干低速だが学習しやすいとされる
    - Java言語：Androidのアプリを作成したり、信用性が重視されるところで利用される

JavaScriptとは

- ブラウザーで見た目を変更したり、ユーザーの操作に合わせて動きを変化することのできる唯一のプログラミング言語
  - ブラウザーの見た目に関わる部分を制御できる
- プログラムを動作させるためには実行するための仕組みが必要
  - JavaScriptの場合、ブラウザーに搭載されているから実行できる

- 現時点ではブラウザーでの表現を変化させることのできる唯一のプログラミング言語
  - 最近のブラウザーではWebAssemblyという仕組みで別のプログラミング言語も動作可能
  - 画面表現を変化させることは不可能

JavaScriptの利用箇所

インタラクション制御

ブラウザーの制御

サーバーサイドのシステム制作

アプリケーションの制御

スマートフォンアプリの作成

JavaScriptの利用箇所

- Webサイトにおけるインタラクション表現
- 利用するタイプのWebサイト（Webアプリ）
  - Gmail、Microsoft365、YouTubeなど多数
- Webサイトを経由して使うものでJavaScriptを使っていないサイトの方が少ない

- サーバー側の処理
  - サーバーサイドプログラムとしての利用

JavaとJavaScript

Java

- 高度なプログラムの作成に利用する
  - オンラインバンキング、ECサイト、業務システムなど
- Androidアプリ
- コンパイル言語

JavaScript

- 比較的短時間動作するプログラム制作などに利用する
  - ブラウザー上でのインタラクション、CSSの変換など
- サーバーサイドシステム
- スクリプト言語

- 双方に全くの関係がない
  - JavaScriptを略してJavaと呼ばないように注意
  - 略すときはJSと略す

コンパイル言語、スクリプト言語

コンパイル言語

- プログラム（ソースコード）を記述
- コンパイラを利用して実行可能な形式に変換する
- 生成されたコード（機械語）を利用して実行する
- C言語、C++、Java、Goなど

動作が高速
作成は大変

スクリプト言語

- プログラム（ソースコード）を記述
- 実行時にコンパイルしつつ、生成されたコード（機械語）を実行する
- Python、Ruby、PHP、JavaScriptなど

動作が低速
作成が容易

JavaScript

- 1995年にNetscapeNavigatorに搭載されたのが最初
- 改良が重ねられることによってバージョンが上がっていく

- 当初は機能不足であまり利用されていなかった
  - 使い方が大きく変わったのはGoogleMapの登場（2005年）
  - 再読込無しにブラウザーを操作できる新しいUIの認識
- 以降JavaScriptを利用したコンテンツが多数登場
  - 全てJavaScriptで制御するタイプのものも多数登場

JavaScript

- JavaScriptの改良時にECMAにより標準化されるように
  - 1997年にECMAScriptとして標準化
    - 以降はECMAScriptがプログラミング言語としての仕様
    - JavaScriptがブラウザーに独自機能を含ませて実装されるときの名前

- 1年に1回程度新しいECMAScriptのバージョンが出る
  - 最新は2025年に登場したECMAScript2026

TypeScript

- JavaScriptでは型指定が存在しない
  - 大規模な開発ではバグが発生しやすく、非効率的である
- コンポーネントという機能に分けてプログラムを作成していく方が大規模なプログラムを開発するときは効率的
  - JSではそういう開発に適した構造を有していない（いなかった）
- MicrosoftがJavaScriptを記述するときに効率的になるように文法を拡張して、2012年に開発したのがTypeScript
  - 一部のブラウザーではそのまま実行できるが、通常はTypeScriptで書いたものをJavaScriptに変換して使う
  - 現在も活発に開発が進み、採用例も多くなっている
  - JSが理解できないと、恐らくTypeScriptは理解しづらい

この講座でのJS

- ECMAScript6（2015）を基本的に利用
  - ES6と表記されることも
  - 少し前のバージョンなので大体のブラウザーで動作する
  - 大きな変更があったバージョン
  - このバージョンを抑えておけば、最新版もある程度理解できる

- jQueryについては扱わない
  - 今はjQueryを新しく覚えるメリットは少ないはず
  - JSが使えれば、jQueryも使うことはできるはず

JSという言語

- 比較的プログラミング言語の中でも複雑
  - 後から機能を追加していったところが多いため、他のプログラミング言語と違う書き方をするところが多い
  - 元々の設計思想がかなり独特
  - 複雑なことをしようとすると、書き方が途端に煩雑になる
  - 参考、サンプルが非常に豊富。探せば直ぐに求めているものが見つかる可能性が高い

- ある程度理解しないとそもそも最初を理解できないというところがある
  - 間違いでは無いが、正しく無いというところが出てくる

プログラミング言語とは

- 1文字でも間違えると動作しなくなる
  - HTMLやCSSは間違えても表示はされる
  - JSは1文字でも間違えると動作しない、最悪ブラウザーが真っ白になる
    - どのプログラミング言語も同じではあるが、入力時注意
    - 打ち間違いはどうしても出るが、できるだけ減らす工夫をする
    - フォントを変更したり、文字を大きくしたり、誤字と思われるところにアンダーラインが引かれるようなエディタを利用してみる

エラーを減らす

- フォントでプログラミング言語向けのフォントを選択する
  - プログラミング言語向けフォントだと日本語のスペースを入れると記号のような文字が入る
  - IとかLとかの区別がしやすい

- 白源
- UDEV Gothic
- PlemoIJP

エラーが起きたとき

- エラーは2種類
  - 1つは文法エラー
    - 作成している段階でツールからエラーとして指し示されることもある
    - 実行すれば直ぐに判明するので直しやすい
  - 1つは論理エラー
    - 一見正しく動いているように見えるのだが、ふとしたときにおかしい動きをする
    - 文法的なエラーは無いが、想定した通りの動作をしない
    - 非常に直しづらい

プログラミングとエラー

- エラーが無いプログラムはまず作成不可能
  - 大体実行すれば文法エラーだの、他のエラーだのが大量に出てくる
  - エラーを直して、再度試して、、の繰り返し
- 最初からきちんと動作するケースの方が少ない
  - エラーメッセージが出てくるだけマシ（それを直せばエラーは解消できるので）
  - トライアンドエラーの世界でもある
- 自分でエラーを解消できる必要がある
  - エラーメッセージが何を指しているかを把握することは重要
  - エラーメッセージは大体英語
  - 生成AIを使ってエラー内容を把握するのも手
    - 全部生成AIに作成させるのは成長しないので、オススメしない

プログラミングとエラー

- 論理エラー
  - 一見動いているが、何か破綻している場合は、
    なぜ破綻しているのかを調べる必要がある
  - そもそもの前提が違う、想定外の影響が及んでいる、
    処理の考え方がおかしかった、など状況よっても原因が異なる
- 論理エラーはエラーメッセージが出ないので、試してみて、
  動作が正常で無かったら調べる必要がある
  - 内部で何が起きているか、値が想定通りかなどを調べて、
    調査をしていく

プログラミング言語

- やりたいことを言語化する能力が必要
  - どういう風にしたら実現可能かを考える能力が必要
  - 文章に落とし込めない、または最終形態が想像付かないならできない
- トライアンドエラーで学んでいく必要性
  - AがダメならB・・、とか考える、調べる必要
  - いろいろな経験を積んでおくと役に立つ
  - 文法を覚えたらいろいろと実際に試してみることが必要
- 自分で手を動かさないとできるようにはならない
  - 本を読んでいても上手く作成できるようにはなりづらい

JSの学習

- 最初は覚えるべき文法が非常に多い
  - 全部覚えてから次に進む、というのは非効率
  - 覚えたもので、できるものを考えて試していく方がベスト
- 新しい書き方や文法を覚えたら改善することが可能
  - もう少し簡単に書けないか、と思ったところは大体簡単に書く方法がある
  - 疑問を感じた方が覚えやすい

JavaScriptの記述

プログラミング言語の数字と文字

- プログラミングでは数字と文字を区別して扱う
- 0と空を区別する

Banana

間違いなく文字

100

数字か文字か？

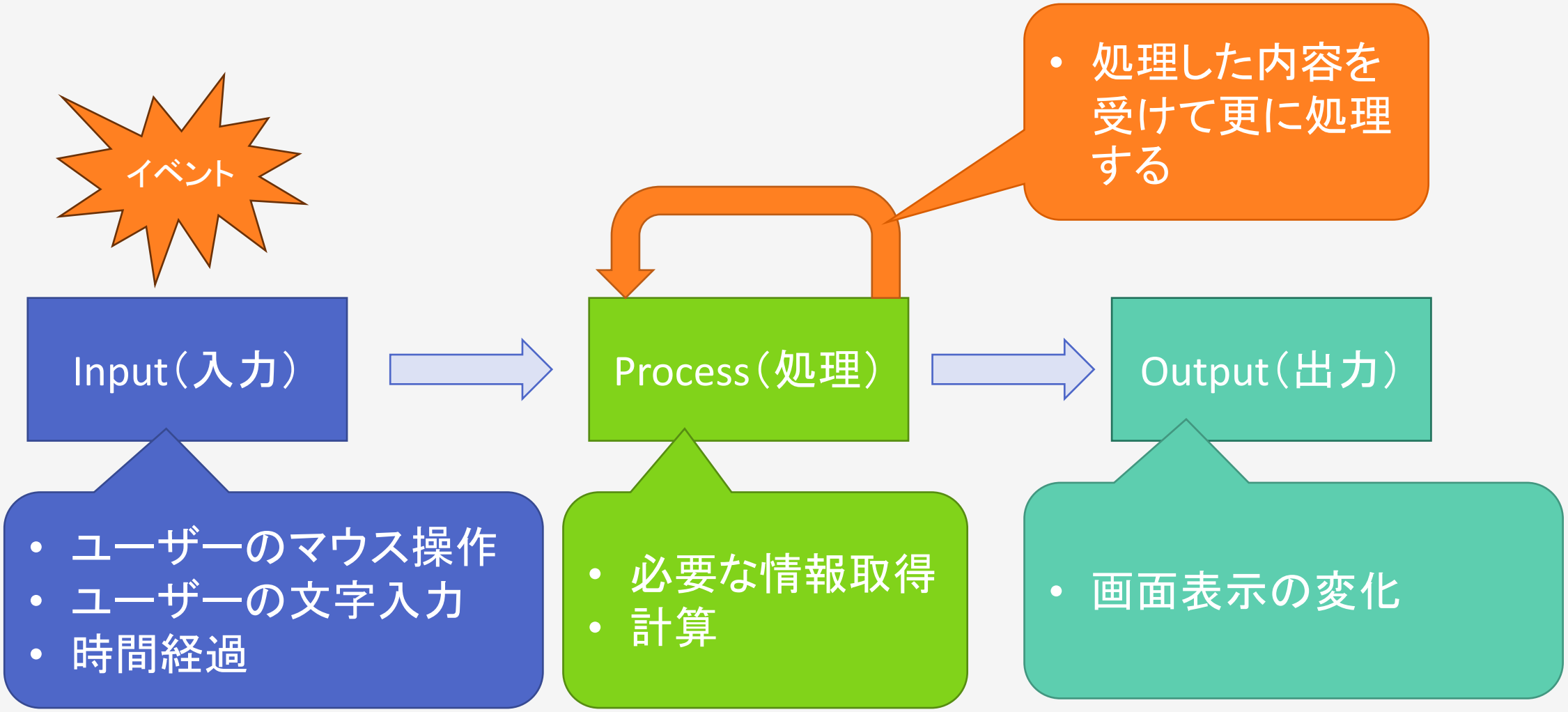
0

0という数字か文字
（空っぽという意味は
無い）

undefined

空っぽ

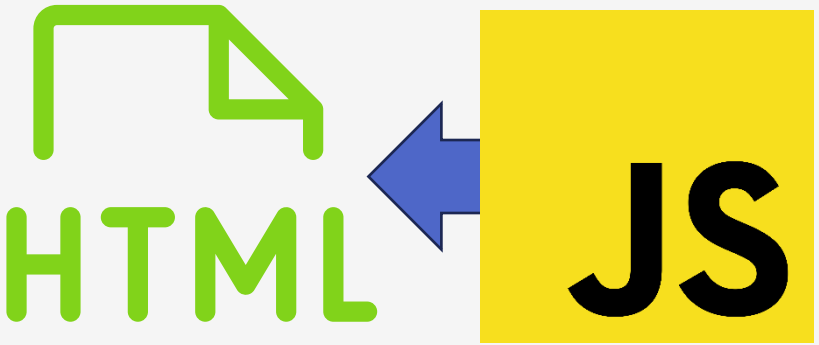
JavaScriptの流れ



- 何らかのイベントが発生したら動く＝イベント駆動

JavaScriptを書いてみる

- HTML内に記述することで実行可能
- <script></script>の間に記述する
  - 別ファイルに記述してもOK
  - 別ファイルの場合は、拡張子をjsとして
    <script src=“〇〇〇.js” ></script>として読み込ませる
- 実行されるタイミングは基本的にブラウザーでHTMLが表示された後
  - ライフサイクル



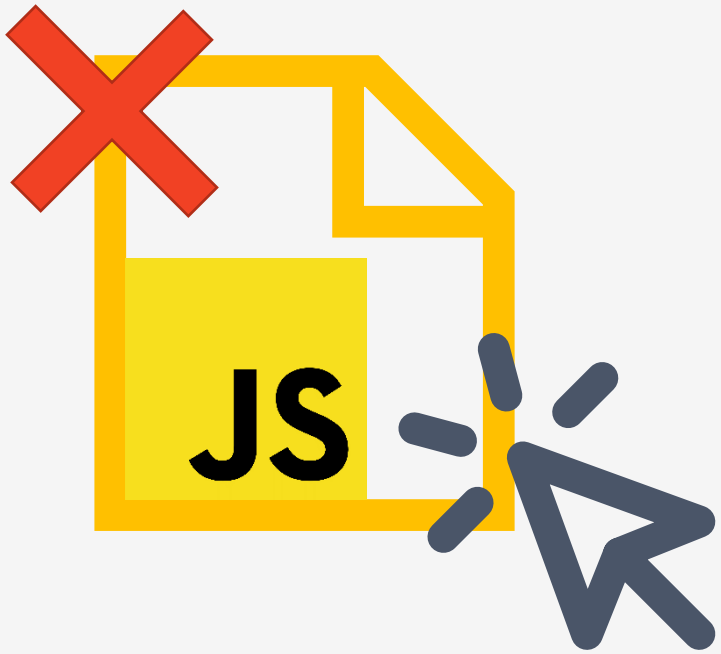
外部JavaScriptファイル

- JSだけを別のファイルに記述してHTMLに読み込ませる方法
  - CSSの読み込ませるのと考え方は一緒
- 読み込みの場所は
  - </body>の直前に記述
  - </head>の直前にdeferまたはasync属性を付けて記述
    - deferはHTMLが読み込まれたら実行する
    - asyncは独立して動作することを保証する

最初のうちはこちらを推奨

```
<script src="sample.js" defer></script>
```

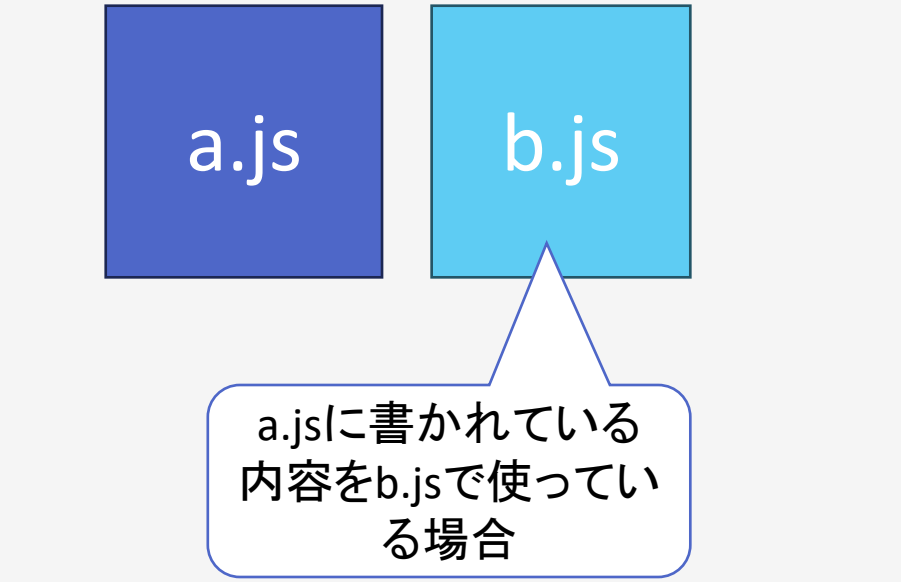
外部JavaScriptファイルの実行



- JavaScriptファイルはダブルクリックしても実行されない
  - 普通のファイルと異なりそれ単体では通常実行できない

外部JavaScript

- 外部JavaScriptは複数のファイルを読み込むことが可能
- scriptタグを並記していけばOK
- 依存関係がある場合は読み込む順番に注意する
  - 例えばjQueryを使っている場合はjQueryを先に読み込む
  - jQueryに依存している部分が他のJavaScriptファイルに出てきたときにjQueryが読み込まれていないとエラーになる




```
<script src="a.js"></script>
<script src="b.js"></script>
```

JSの記述

- 基本的には英数字および記号を利用する
  - 全角はシングルクォーテーション、ダブルクォーテーションおよびバッククォート内、コメント部分だけ利用可能
    - 実際にはJSは日本語も文法に沿えば利用可能だが、通常使わない
  - 先頭に記号は使わない
    - 特殊な意味を持っているものが多い

- コメントを日本語で記述した後でそのまま書いてエラーになることが多いので注意
  - スペースが全角だとエラーになるが探しづらい

JS

- JSでは変数、メソッド、イベントなどが用意されている
  - 変数＝一時的に保存しておくための場所。名前を付けることでいろいろな内容を保持可能
  - 関数＝いろいろな処理をまとめて記述したもの。後で呼び出すことにより実行することができる
  - 制御＝プログラムを分岐させたり繰り返したりして、制御を行う
  - メソッド＝JavaScriptで利用することができる機能のこと。1つ1つでは大したことはできない
    - 組み合わせることで処理をしていくことが可能
  - イベント＝ユーザーの行動（インタラクション）など

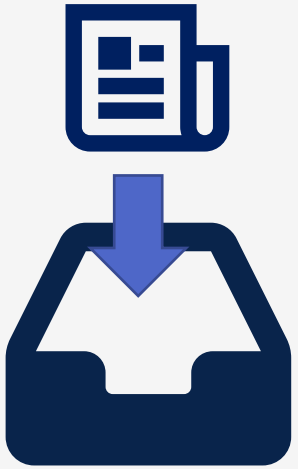
JSのコメント

- プログラムの動作に影響しないメモなどを記述するときに利用する
  - プログラムの一部を無効にするときにも利用する
    ＝コメントアウト
  - 単一行のコメントと複数行のコメントが存在
  - コメントは日本語を記述することが可能

```
//  1行がコメントとして扱われる
/*
    複数行がコメントとして扱われる
*/
```

JSの基本的なもの

- 変数
  - 何かの値やオブジェクトを格納しておくもの
  - JSの場合、いろいろなものを格納可能
    - オブジェクトも格納可能
  - 先頭にletを付ける（変数宣言）
    - スペースを開けて変数名を指定（通常1文字目は英小文字）
    - 一部の単語は予約語として利用できない
      - 良く使うような単語は比較的使えない



```
let a = ‘JavaScript’;//aに”JavaScript”という文字列を格納
let b = 123;//bに123という数値を格納
```

変数への値

- シングルクォーテーションかダブルクォーテーションで括ると文字として扱われる
  - シングルクォーテーションを推奨
- 数字として格納するときは数字だけ記述する
- 文字をシングルクォートやダブルクォート、バッククォートで括らない場合は変数として扱われる

型指定

- JavaScriptは動的型付け言語
  - 型というのを動的にJavaScriptの実行環境が設定する
  - 型：変数にどのような種類のデータが入っているかを示すもの
    文字型、数字型などが存在する
  - JavaScriptの場合、実行環境が自動的に判断して処理して
    くれる
    - 便利とも思えるが、現在は静的型付け言語（自分で型を指定する）のが
      流行っている
    - TypeScriptは静的型付け言語

状態と副作用

- プログラムが動いている状態で保持している今の値のことを状態と呼ぶ
  - 変数の値や画面に表示されている内容なども状態として扱われる

- 副作用
  - 関数や処理が「値を計算して返す」以外の影響を与えるものを副作用と呼ぶ
    - 画面を書き換える
    - 変数の値を書き換える
      などが副作用になる

文字列連結

- 文字同士を繋げるときには「+」記号を利用する

```
let a = ‘Java’ + ‘Script’;//JavaScriptが格納される
```

- 変数同士（中身が文字列の場合）を繋げるときも「+」記号を利用する

```
let b = ‘Java’;
let c = ‘Script’;
let a = b + c;//JavaScriptが格納される
```

文字列連結

- 文字と数字を繋げるときは注意

```
let a = 100 + ‘200’;//300にはならない。100200になる
```

- 数字同士の場合

```
let a = 100 + 200;//300になる
```

- 「＋」の記号だけ変数の中身または文字列や数字など結合するものによって動作が変わる点に注意

計算、代入

- =の記号は等号ではなく、左辺に右辺を代入となる

```
let a = 100;
```

変数aが100ということではなく、変数aに100を代入する

- 他の算術子は通常通り

```
let a = 100 + 20;
let b = (100 + 20) * 3;
let c = 120 / 3 + 5
```

120が代入される

360が代入される

45が代入される

変数

- 通常は変数宣言を行って利用する（宣言しなくてもOKだが、最近は警告が出ることが多い）
  - 先にどういう変数を使うかを教えてあげておく
  - 先頭にletを付ける（変数宣言）

```
let a;//aという変数を利用する（宣言のみ）
let b = 1;//bという変数を利用すると宣言し、1という数値を格納
let c = ‘js’//cという変数を宣言し、jsという文字列を格納

a = 4;//宣言済みの変数aに4を格納
```

画面に描画

- 変数に格納しているだけでは画面には表示されない
- 表示するためには命令を書いて表示する

```
let name = ‘JavaScript’;
document.write(name);

document.write(‘Programing Language’);
```

変数にデータを格納して

変数を表示

変数に格納しないで表示

```
window.alert(‘JavaScript’);
```

アラートダイアログを表示

テンプレート文字列

- テンプレートリテラルとも呼ばれる
- バッククォートで文字列を括って表現する
  - バッククォート内で${式}を記述して展開させることが可能
  - 例えば、${name}などで変数を展開させることが可能
    ${price * 1.1}のように式を書くことも可能であり、
    また文字列連結記号は不要

```
let name = ‘みかん’;
let price = 200;
let msg = `今日の${name}の値段は${price * 1.1}円です`;
```

特殊な値

- undefined⇒初期化されていない変数の値

```
let a;
window.alert(a);//undefined
```

- null⇒undefinedとほぼ同じだが値を指定しないと出てこない

```
let a = null;//nullを代入
window.alert(a);//null
```

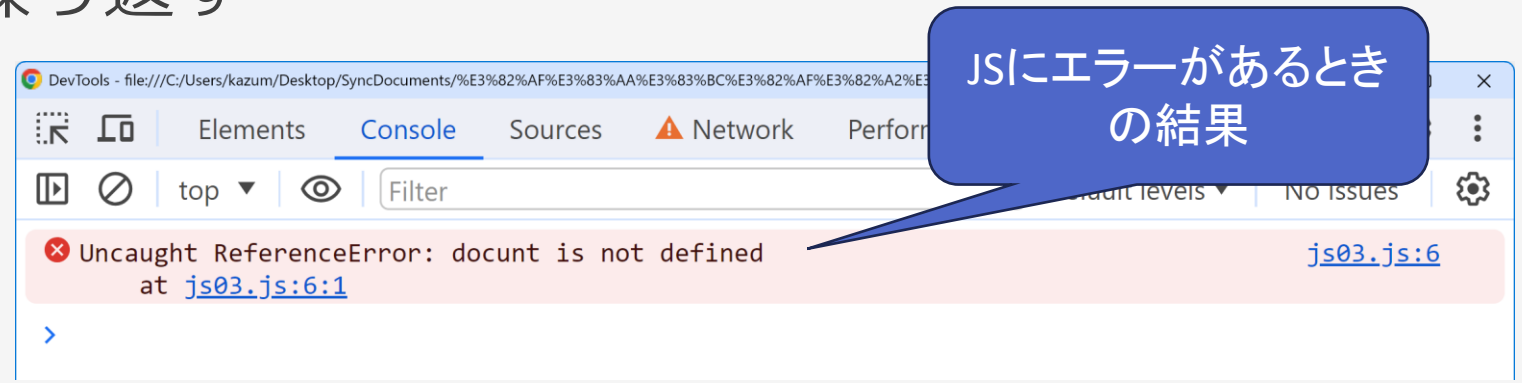
特殊な値

- true ， falseという値
  - シングルクォーテーションなどを付けないで表す
  - 真、偽という値を表す（Bool値）
- JSのメソッドを使ったときの結果として得られることがある
  - ON、OFFなどを示すときに変数として使われることもある

```
let a = true;
```

エラーが起きて動作しない

- ChromeのDevToolsを使う
  - ブラウザーウィンドウ内で右クリックして「検証」を開く
  - Command+Option+iキー（mac）
- consoleタブを開くとJSでエラーがあればエラー内容が表示される
  - Chromeに拡張機能を入れているとそのエラーが出ることもある点に注意
- 大まかな行数も出るので、その辺りを入念にチェック
- 大量に出ても1つだけ直せば消えることも多いので1つずつ修正して確認、を繰り返す



エラーの解消

- エラーとなっているところを直して問題無く動けばOK
- スタックトレースも出てくる
  - どの関数を順番に呼び出してエラーとなる部分に来たかの履歴
- 直しても想定外の動きをする場合は、変数の値などを確認する必要がある
  - alert(変数) などで変数の中身を確認できるが、配列などの場合は確認できない

```
console.log(変数名);
```

  - console.log(変数名) を書くことで、DevToolsのconsole画面に変数の中身を表示することが可能

ありがとうございました。

また次回。